AUTOMATISOITU OSAKETIETOJEN KERUUJÄRJESTELMÄ

Jonni Tuki - Toisen vuoden data-analytiikan opiskelija

Projektisuunnitelma ja toteutus

Sisällysluettelo

ALKUSANAT **2**

1. GIT repositorio . . . 2
2. Projektin suunnitelma . . . 2
    2.1 Tavoitteet . . . 2
    2.2 Järjestelmät . . . 2
3. Tekoäly osana projektia . . . 3

TOTEUTUS **3**

1. Palvelimen luominen Linux-järjestelmälle . . . 3
    1.1 Alustus . . . 3
    1.2 Ratkaisu ja lopullinen toteutus . . . 3
    1.3 Automaattisuudesta . . . 5
2. SQL-tietokanta . . . 5
    2.1 Vaaditut taulut . . . 5
3. Pääohjelman toteutus . . . 6
    3.1 Yhteyskokeilu . . . 6
    3.2 Osakkeiden haku . . . 6
    3.3 Datan haku ja puhdistus . . . 7
    3.4 Tietokanta tallennus . . . 8
    3.5 Gmail yhteenveto . . . 8
    3.6 Virheenkäsittely . . . 9

TOTEUTUKSEN JÄLKEEN **10**

1. Muokkaukset . . . 10
2. Potentiaali . . . 10
3. Itsearvio . . . 10

ALKUSANAT

Tämä projekti on jatkoa Tampereen yliopiston *DATA.STAT.210*-kurssilla toteutetulle empiiriselle työlle. Työn taustalla vaikuttavat kurssityön tulokset, saatu palaute sekä henkilökohtainen kiinnostus rahoitusmarkkinoihin ja energiamurrokseen. Tavoitteena on syventää aihealueiden tietämystä tulevaisuuden työelämän tarpeita varten. Projekti on toteutettu aikavälillä 6/2026-11/2026.

1. GIT repositorio

Varsinainen ohjelma löytyy tiedostosta *stocksearch.py*, seuraavan linkin takaa: https://github.com/JTuk1765/projects.

2. Projektin suunnitelma

2.1 Tavoitteet

Projektilla on kaksi tavoitetta, jotka on tarkoitus toteuttaa eri aikajänteillä:

1. **Ensiarvoinen tavoite:** Automatisoidun osaketietojen keruujärjestelmän luominen. Järjestelmä hakee määriteltyinä ajankohtina valittujen osakkeiden tiedot ja tallentaa ne SQL-tietokantaan. Lisäksi ohjelma lähettää käyttäjälle päivittäisen tiivistelmän tiedonkeruun tuloksista. Varsinainen projekti katsotaan valmiiksi, kun järjestelmä toimii stabiilisti ilman manuaalista väliintuloa.
2. **Pitkän aikavälin tavoite:** Kerätyn aineiston pohjalta tehtävä osakeanalyysi ja sellaisten kriteerien tunnistaminen, joiden avulla rahoitusmarkkinoilla on mahdollista saavuttaa ylituottoa. Tämä vaatii pitkäkestoista datan keruuta sekä tallennusta ilman ongelmia. Varsinainen projekti päätetään ensimmäisen tavoitteen onnistumiseen, eli automatisoidun järjestelmän käynnistymiseen ja sujuvaan pyörimiseen.

Teknisenä tavoitteena on vahvistaa Python- ja SQL-osaamista, tietokantojen suunnittelua sekä datanhallintaa. Lisäksi projektissa perehdytään versionhallintaan (GitHub), komentorivin käyttöön sekä Linux-ympäristön automatisointiin.

2.2 Järjestelmät

Ohjelmointikieli (Python): Toteutuskieleksi valittiin Python sen laajojen kirjastojen ja soveltuvuuden vuoksi.

Tietokanta (Supabase / SQL): Tiedot tallennetaan SQL-tietokantaan. Tietokantapalveluna käytetään Supabasea, joka tarjoaa helpon integroitavuuden Python-ympäristöön.

Versionhallinta (GitHub): GitHub toimii projektin versionhallinta- ja varmuuskopiointialustana. Se mahdollistaa myös sujuvan koodin siirron eri käyttöjärjestelmien välillä.

Palvelinympäristö (Linux): Järjestelmän ajoympäristönä toimii Linux-pohjainen palvelin. Linux valittiin sen keveyden, suorituskyvyn ja luotettavuuden ansiosta, mikä mahdollistaa ohjelman ajamisen myös vanhemmalla laitteistolla.

3. Tekoäly osana projektia

Projektissa on käytetty Googlen Gemini 3.6 Flash ja Pro malleja. Tekoälyn käytöstä kerrotaan projektissa tapauskohtaisesti. Lisäksi ohjelman koodissa on merkitty ne funktiot, joissa on käytetty tekoälyä joko apuna tai suoraan funktion muodostamiseen.

Tekoälyä on käytetty erityisesti vaikeimpien funktioiden muodostuksessa. Syynä tähän on entuudestaan tuntemattomien kirjastojen käyttö, joiden opettelu on vielä tässä vaiheessa opintoja päätetty jättää hieman vähemmälle ajan ja resurssien säästämiseksi.

TOTEUTUS

1. Palvelimen luominen Linux-järjestelmälle

1.1 Alustus

Tavoitteena on luoda tietokoneelle täysin automaattinen järjestelmä, joka käynnistyy haluttuina viikonpäivinä, suorittaa pääohjelman ja sitten sammuttaa tietokoneen. Halutut viikonpäivät ovat maanantai, keskiviikko ja perjantai. Näihin päiviin päädyttiin ajantasaisen datan saamiseksi.

Järjestelmän luominen aloitettiin asentamalla vanhalle tietokoneelle Linux järjestelmä. Järjestelmän asentamisessa ei ollut suurempia haasteita. Asennuksen jälkeen luotiin `server` kansio `työpöydälle`. Tämä kansio liitettiin Github repositorioon, joka oli jo aiemmin liitetty myös varsinaisen pääkoneeni vastaavaan kansioon.

Seuraavaksi lähdin luomaan varsinaista automaatiota. Aloitin ensimmäiseksi kysymällä tekoälyltä, mitä käytännön vaihtoehtoja on automaation toteuttamiseen. Vaihtoehtojen perusteella päädyin siihen, että tietokoneen pitäisi toimia seuraavalla tavalla:

1. Kone on kansi kiinni, sammuneena, herätys on asetettu edellisen käynnistyksen yhteydessä haluttuun aikaan ja viikonpäivään.
2. Tietokone käynnistää itsensä. Järjestelmä odottaa 2 minuuttia, jotta nettiyhteys muodostuu.
3. Järjestelmä suorittaa pääohjelman
4. Lasketaan seuraavan käynnistyksen aikataulu
5. Sammutetaan tietokone.

Fyysisesti käyttäjän täytyisi ainoastaan kytkeä järjestelmä verkkovirtaan säännöllisesti, jotta virta ei pääse loppumaan kesken vanhasta kannettavasta.

Ongelma

Heti alussa tuli ongelma järjestelmän heräämisen kanssa. Kone kyllä sammuu mutta sen käynnistyminen oikeaan aikaan ei onnistu kansi auki eikä kiinni. Lopputulos olikin se, että järjestelmä ei tue automaattista käynnistystä sammutuksesta.

1.2 Ratkaisu ja lopullinen toteutus

Koska sammutusta ei voida tehdä, ja järjestelmän halutaan pysyvän mahdollisimman automaattisena, päädyin ratkaisuun jossa kone sammutetaan vain viikonlopuksi. Muutoin kone vietäisiin aina lepotilaan ajon jälkeen. Muutosten myötä ohjelma toimii seuraavasti:

1. Kone käynnistetään maanantaina manuaalisesti ja ajetaan linux komennolla “`sudo bash`” , ohjelma *ajo.sh* joka antaa käyttöjärjestelmälle ohjeet tiedonkeruun aikataulutuksesta.
2. Ohjelma laskee seuraavan käynnistyksen ajankohdan ja palaa lepotilaan
3. Käyttöjärjestelmä herättää koneen lepotilasta seuraavana ajohetkenä kello 17
4. Suorittaa tiedonkeruun
5. Laskee uuden herätys ajankohdan ja palaa lepotilaan

Ohjelma toistaa tätä silmukkaa viikonpäivinä maananatai, keskiviikko ja perjantai. Perjantain ajon jälkeen kone sammuttaa itsensä automaattisesti. Tämä tehdään tietokoneen eliniän parantamiseksi ja akun säästämiseksi. Maanantaina kone täytyy käynnistää manuaalisesti `sudo bash <tiedostopolku>` komennolla.

Järjestelmää ja sen lepotila - käynnistys syklejä säädellään ajo.sh tiedostolla, jonka sisältö näyttää seuraavalta:

```
# ajo.sh tiedosto joka vastaa tiedonkeruun aikataulusta.

#!/bin/bash

# 1. ENSIMMÄINEN LEPOTILA
# Käynnistettäessä maanantaina käsin, kone menee lepotilaan klo 17:00 asti
MAANANTAI_AJO=$(date -d "17:00:00" +%s)
rtcwake -m mem -t $MAANANTAI_AJO

# 2. PÄÄSILMUKKA (Ma -> Ke -> Pe)
while true; do
    # suoritetaan skripti joka tarkastaa päivämäärän ja kellonajan
    # Suoritetaan Python-skripti herätyksen jälkeen klo 17:00
    .../kello.py
    .../stocksearch.py

    PAIVA=$(date +%u)

    if [ "$PAIVA" -eq 1 ] || [ "$PAIVA" -eq 3 ]; then
        # Maanantaina (1) ja Keskiviikkona (3) nukahtaa 2 vuorokaudeksi klo 17:00 asti
        UUSI_AIKA=$(date -d "2 days 17:00:00" +%s)
        rtcwake -m mem -t $UUSI_AIKA
    elif [ "$PAIVA" -eq 5 ]; then
        # Perjantaina (5) odotetaan 20s ja sammutetaan kone kokonaan
        sleep 20
        shutdown -h now
        break
    else
        echo "Skripti suoritettiin muuna kuin Ma/Ke/Pe -päivänä ($PAIVA). Lopetetaan."
        exit 1
    fi
done
```

Tiedostossa mainittu *kello.py* ohjelma on tehty tarkastamaan kellonaika ja päivämäärä. Tiedosto on tehty siksi, koska koneen kannen avautuessa ohjelman laskuri menee sekaisin ja olettaa, että kello on jo halutun verran. Tästä seuraa tiedonkeruun välitön aloittaminen kannen avautuessa. *kello.py* ohjelma yksinkertaisuudessaan laskee tiedonkerun aloitusajan uudelleen ja aloittaa tauon joka kestää tähän asti. Koneen asetuksista on estetty lepotila, horrostila sekä sammuminen.

1.3 Automaattisuudesta

Ohjelman automaattisuuden suhteen ei päästy aivan siihen tulokseen mitä alunperin suunniteltiin. Syynä tähän oli aiemmin mainittu este sammuneen koneen herätyksessä.

Jos ohjelmaa haluaisi ajaa nykyisellä laitteistolla automaattisempaan suuntaan, pitäisi tehdä kaksi olennaista muutosta. Perjantain sammuttaminen täytyisi peruuttaa ja muuttaa lepotilaan ajoksi. Näin tietokone ei hukkaisi aikataulua ja sykli pyörisi katkeamattomana erilliseen sammutukseen saakka.

Nykyisellään kannettavan tietokoneen lataaminen vähentää automaattisuutta ja lisää fyysisiä työvaiheita. Tästä päästäisiin eroon joko ohjelmiston muokkaamisella ja latausrajoituksia asettamalla. Vaihtoehtoisesti myös fyysinen ajastin latauspäässä voisi hoitaa ongelman. Henkilökohtaiseen käyttöön kuitenkin nykyinen malli jossa konetta ladataan säännöllisesti parin päivän välein on ollut aivan riittävä ja automaattinen.

Nykyisen systeemin työtä vaativat vaiheet ovat seuraavat: kerran viikossa terminaalissa yhden komennon ajaminen, virtajohdon kytkeminen, kannettavan kannen avaaminen. Vaikka tavoite täydestä automaattisuudesta jäi täyttämättä, pidän nykyistä tilannetta tarpeeksi automaattisena, jotta voidaan todeta tavoitteen täyttyneen.

2. SQL-tietokanta

2.1 Vaaditut taulut

Alusta asti oli selvää että tiedot on järkevää tallentaa sql tietokantaan, koska se oli kielenä jo entuudestaan tuttu. Tietokannan rakentaminen alkoi sisällön kartoittamisesta. Päädyin siihen että tarvitsen kolme taulua. *mean, stock_data* sekä *positions*.

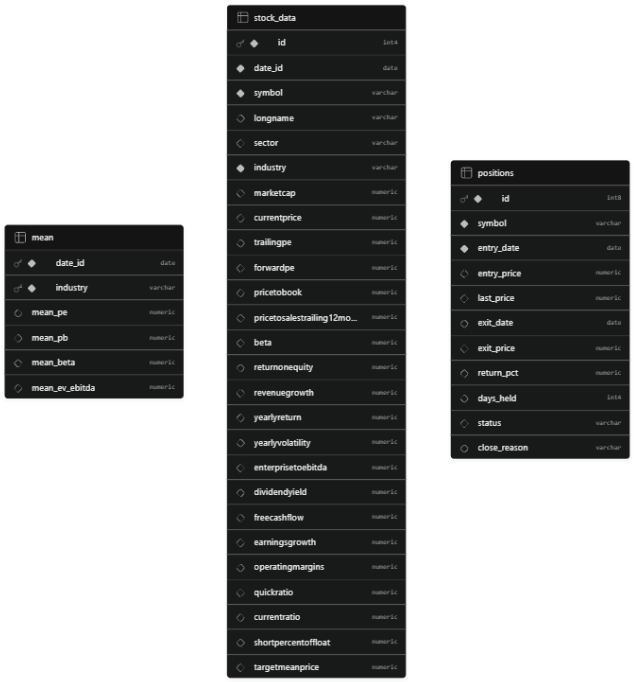
Taulut eivät ole toisiinsa liitoksissa mitenkään, mutta päivämäärän perusteella voidaan yhdistää tietoa taulujen välillä.

stock_data - taulu: Tämä taulu toimii kaiken datan tallennuspaikkana. Jokainen osake tietoineen tallennetaan tänne. Tauluun päätyy osakkeen tiedot riippumatta sen datasta.

mean - taulu: Tauluun tallennetaan toimiala kohtaisesti keskiarvo P/E, P/B, Beta ja EV/EBITDA.

positions - taulu: sisältää osakkeet jotka ovat kriteerien pohjalta päätyneet aliarvostetuiksi. Näistä kriteereistä lisää luvussa 3.

Alla olevassa kuvassa on esitettynä sql tietokannan rakenne.



3. Pääohjelman toteutus

Tässä luvussa puretaan osiin pääohjelmaa olennaisilta osin. Esimerkeissä olevia koodeja on tiivistetty paikoittain paljonkin jotta luettavuus ja selkeys säilyy. Koodin voi kokonaisuudessaan katsoa luvun *ALKUSANAT* Github linkistä.

3.1 Yhteyskokeilu

Funktio connection vastaa yhteyskokeilusta ja se tarkistaa testi osoitteen https://1.1.1.1 kautta että internet yhteys on muodostunut ja että tiedonkeruu voidaan täten aloittaa. Funktio yrittää tätä testiä korkeintaan 5 yritystä, pitäen välissä 20 sekunnin tauon. Taukojen ja useiden yritysten tehtävänä on varmistaa, että laitteella on hyvin aikaa yhdistää itsensä WLAN yhteyteen.

3.2 Osakkeiden haku

Osakkeiden nimet ja alustava suodatus

Osakkeiden haku ja alustava suodattaminen suoritetaan yhdellä funktiolla `stock_list`. Funktio muodostaa yhteyden Nasdaq API palveluun, jonka kautta se saa tiedot kaikista yhdysvaltalais osakkeista. Tämän jälkeen funktio suorittaa alustavan rajauksen toimialojen perusteella ja kirjaa sopivien osakkeiden symbolit eli lyhytnimet niille varattuun tiedostoon.

Osakelistat haetaan julkisesta NASDAQ API palvelusta, koska se on ilmainen, virallinen ja sallii osakkeiden haun tehokkaasti yhdellä kyselyllä. Tämä nopeuttaa prosessia, koska ei tarvitse huolehtia palvelimen ylikuormituksesta tai prosessin pitkittymisestä. Haussa kestää aina vain joitain sekuntteja.

Haun jälkeen halutaan tehdä alustava rajaus myös toimialalle ja sektorille. Käytetään hakutekniikkaa, jossa ensiksi poistetaan sellaisia yhtiöitä joiden toimialasta tai sektorista löytyy kiellettyjä avainsanoja, eli sellaisia

termejä jotka viittaavat vääränlaiseen yhtiöön. Sen jälkeen tehdään sama haku halutuilla toimialoilla. Koska tavoitteena seurata uusiutuvan energian yhtiöitä, jakautuu suodatin listat seuraavasti:

```
vältettävät toimialat = [öljy, kaasu, fossiiliset,...]   # uusiutumattomat energiat
    halutut toimialat = [uusiutuva, aurinko, puhdas energia,...]   # uusiutuvat
```

Lajittelussa hakusana `uraani` on laitettu haluttuihin toimialoihin, vaikka se virallisesti onkin uusiutumaton, tämä johtuu omasta kiinnostuksesta ydinvoimaan ja ydinvoiman puhtaudesta.

Suodatuksen jälkeen osakkeiden määrä on tyypillisesti saatu pudotettua yli 10 000 kappaleesta noin 300 osakkeeseen. Syy datan puhdistukselle jo tässä vaiheessa on `yfinance` palvelimen ylikuormittuminen, jonka olen havainnut tapahtuvan hyvin nopeasti jo satojen osakkeiden kohdalla. Kymmenen tuhannen osakkeen käsittely olisi vaatinut ohjelmalta pitkiä taukoja, ja siltikin suoritus olisi todennäköisesti keskeytynyt. Noin 300 osakkeen käsittelyn pystyy turvallisesti suorittamaan peräkkäin ilman että palvelu ylikuormittuu ja aikakatkaisee käyttäjän IP-osoitteen.

3.3 Datan haku ja puhdistus

Osake datan keräys

Tietojen keruusta vastaa ohjelmassa kaksi funktiota: `workers` sekä `gather_data`. Näistä ensimmäinen on vastuussa prosessin aloittamisesta, se määrittelee tiedonkeruuseen käytetyn ajan ja säätelee taukoja. `Workers` funktio suorittaa kerrallaan useita päällekkäisiä kertoja funktiota `gather_data`, jotta prosessi olisi nopeampi. Tämä toteutetaan `concurrent.futures` kirjaston `ThreadPoolExecutor` avulla seuraavasti:

```
from concurrent.futures import ThreadPoolExecutor

# päällekkäisten toistojen määrä = 5, erä = 300 osaketta, tauko = 180 sekunttia
# <- for silmukka joka vastaa osakkeiden läpikäynistä

  # suorittaa gather_data funktiota
  with ThreadPoolExecutor(max_workers=num_workers) as executor:
              future_to_stocks = {executor.submit(gather_data,
              s, infos, error_file): s for s in chunk}

  # <- jos erän jälkeen yhä käsittelemättä osakkeita --> tauko
```

Kun `workers` funktio kutsuu `gather_data` funktiota, hakee funktio `yfinance` kirjaston avulla tiedot kaikista jäljellä olevista osakkeista. Haluttujen tietojen lisäksi lasketaan tässä kohtaa myös vuosituotto eli `yearlyReturn`, sekä vuosittainen volatiliteetti eli `yearlyVolatility`.

datan puhdistus

Kun data on kerätty, siitä erotellaan `sorter` funktion avulla uudestaan väärät toimialat pois. Aluksi tehdään suodatus apufunktiolla `is_renewable_company`, joka varmistaa uudestaan että toimialat ovat varmasti uusiutuvan energian yhtiöitä `longBusinessSummary` sarakkeen avulla. Sarake sisältää tekstimuotoisen selostuksen siitä, mitä yhtiö tekee. Sarakkeen alkioista etsitään avainsanoja jotka vahvistavat sen, pysyykö yhtiö listalla. Käytännön toteutus toimii seuraavalla tavalla:

```
if 'longBusinessSummary' in df.columns:
        df = df[df['longBusinessSummary'].apply(
            lambda x: is_renewable_company(x, wanted, unwanted))]
            # jossa wanted = [] sisältää uusiutuvan energian avainsanoja
            #     unwanted = [] fossiilisiin viittaavia sanoja
```

Näiden lisäksi rajataan pois sellaiset yhtiöt joiden markkina-arvo on yli 5 miljardia dollaria. Näin säilytetään datan vertailukelpoisuus.

Toimiala ja markkina-arvo suodatuksen lisäksi haluttujen aliarvostus kriteerien perusteella jaetaan yhtiöt aliarvostettuihin ja muihin yhtiöihin. Aliarvostus määräytyy p/e, p/b ja beta arvojen perusteella. Aliarvostuksen saavuttamiseksi yhtiön pitää täyttää 2/3 ehdoista päästäkseen aliarvotus listalle.

3.4 Tietokanta tallennus

Tämä funktio pääsi kasvamaan todella pitkäksi, koska jokainen tauluun tallennus tehdään tämän sisällä. Jatkoa ajatellen jokaisen taulun voisi tehdä ommassa funktiossaan, tai yhdessä funktiossa johon syötetään tarvittavat tiedot.

Funktio `sql_database` muuttaa sekä koko datan dataframen, että aliarvostetun datan dataframen sanakirjaksi. Tämän jälkeen tallennetaan tauluhin niihin kuuluvat tiedot. Taulut `mean` ja `stock_data` ovat suoraviivaiset, niihin tallennetaan sen päivän data.

Taulu `positions` vaatii enemmän tarkastelua. Se tarkastaa löytyykö osaketta jo positions ltaulusta. Jos löytyy se tarkastaa onko osake ollut yli vuoden listalla, mikäli on, se myydään. Tämä on tietoinen valinta ja liittyy strategiaan, jossa pääoma täytyy saada kiertoon korkeintaan vuoden jälkeen, koska tarkoituksena on hakea tuottoa nopeammissa sykleissä. Mikäli osake löytyy listasta, sen hinta päivitetään. Mikäli osaketta ei löydy, luodaan sille uusi rivi. Jos positions tauluun jää osake jolle ei ole päivältä uutta tietoa, tulkitaan aliarvostuksen päättyneen ja osakkeelle kirjataan päätöshinta. Kaikille osakkeille jotka poistuvat seurannasta, kirjataan lyhyt syy poistumiselle. Poistumisen yhteydessä osakkeille lasketaan myös poistumisprosentti, jota se on tuottanut.

3.5 Gmail yhteenveto

Yhteenveto lähetetään jokaisen ajon jälkeen sähköpostiin, se sisältää tiedoston jossa kerrotana perustiedot ajosta ja uudet aliarvostetut osakkeet. Esimerkki sähköpostista päivältä 23.10.2026, kyseessä perjantain ajo, joka on toteutunut poikkeuksellisena aikana, johtuen luvussa 1.3 mainitusta ongelmasta automaattikäynnistyksen suhteen (omat kiireet estäneet koneen kannen avaamisen).

```
--AJON YHTEENVETO--
päivämäärä: 2026-10-21
kellonaika: 17:01:15


HAUN TIEDOT:
Aloitusaika: 2026-10-21 16:59:58
Lopetusaika: 2026-10-21 17:01:15
Kesto sekunteina:  76.54 s

Löydettyjä osakkeita: 281 kpl
Sopivat osakkeet:  2 kpl
Virheiden määrä:   0 kpl

--- KESKIARVOT (Sopivat osakkeet) ---
Markkina-arvo ka:  2,560,641,408 $
P/E keskiarvo:     10.66
P/B keskiarvo:     0.66
Beta keskiarvo:    0.33

--- SOPIVAT OSAKKEET ---
Nimi: Pampa Energía S.A. | SMBL: PAM | Toimiala: Conglomerates | Hinta: 78.06
Nimi: XPLR Infrastructure, LP | SMBL: XIFR | Toimiala: Utilities - Renewable | Hinta: 9.82

Osakkeet on tallennettu Supabase tietokantaan.
JONNI TUKI (automaattinen ohjelma)
```

Lisäksi toimitetaan myös *errors.txt* mikäli johonkin virheeseen on päädytty. Tämä on tapahtunut kerran testivaiheessa, jolloin virhetiedosto näytti seuraavalta

```
Virhe Supabase-tallennuksessa (data-taulu): {'message': "Could not find the 'fullexchangename'
column of 'stock_data' in the schema cache", 'code': 'PGRST204', 'hint': None, 'details': None}
```

3.6 Virheenkäsittely

Mikäli ohjelman suorituksen aikana tapahtuu mikä tahansa olettamaton asia, kuten yhteysvirhe, palvelinvirhe tai virhe osakkeen tiedoissa, tallennetaan kyseisen vian selitys vikatiedostoon. Tiedosto tallennetaan paikallisesti palvelin koneelle, sekä lähetetään vikatilanteiden ilmestyessä myös käyttäjälle. Poikkeustilanne jossa tiedostoa ei lähetetä käyttäjälle, on virhe jossa internet yhteyttä ei pystytä muodostamaan, jolloin myöskään sähköpostin lähetys ei onnistu.

Yleisimmät virheet koskevat internet yhteyden muodostamista, palveluiden kuten Yfinance ja NASDAQ API ylikuormittumista tai palvelin muutoksia sekä puutteellisia tietoja yksittäisissä osakkeissa. Ohjelma osaa ohittaa kaikki yleisimmät virhetilanteet, joten ohjelman kaatuminen kesken ajon ei ole todennäköinen.

On myös teoriassa mahdollista, että Linux ajossa kohdataan jokin ongelma ja ohjelman suorittaminen ei koskaan käynnisty. Näissä tilanteissa virhetiedostoon ei päästä kirjoittamaan mitään syytä, tällaisen tilanteen tunnistan saapumattomasta sähköpostiviestistä. Näissä tilanteissa täytyy virhetarkastelu tehdä manuaalisesti Linux terminaalin kautta.

Tätä yhteenvetoa kirjoittaessa ei noin 2 kuukauden ajalta, jolloin ohjelma on ollut käynnissä 3 kertaa viikossa, ole huomattu yhtäkään virhettä tavallisten ajojen yhteydessä.

TOTEUTUKSEN JÄLKEEN

1. Muokkaukset

Olen tehnyt ohjelman tekemisen jälkeen hieman analyysiä datasta ja tehnyt sen perusteella muokkauksia, jotta ulos tuleva data olisi itselleni osuvampaa. Toteutus on kuitenkin pysynyt täsmällään samana. 24.10 Jälkeen on alettu tallentamaan osaketietoa laajemmin uusiutuvan energian osakkeista. Tätä ennen tietoa on tallennettu vain kahdelta toimialalta. Laajennus tehtiin, jotta voidaan tehdä kattavampaa analyysia koko sektorista ja näin saada luotettavampaa ennustusta.

2. Potentiaali

Kuten tavoitteissa mainittiin, on pitkän ajan tavoite kehittää omaa sijoitusstrategiaa. Käytännössä tälle on monia mahdollisuuksia. Tällä hetkellä on kiinnostusta kehittää analyysia esimerkiksi liittämällä säätieto dataa hinnan selittäväksi muuttujaksi. Myös fossiilisen energian tuotantomäärien vaikutusta uusiutuvan energian yhtiöihin on halukkuutta testata.

Laajenettu data tarjoaa myös mahdollisuuksia etsiä eroavaisuuksia voittavien ja häviävien osakkeiden välillä. Tästä saadun informaation perusteella voidaan myöhemmin esimerkiksi aliarvostuksen kriteereitä muuttaa.

3. Itsearvio

Projektin arvioisin onnistuneen kohtalaisen hyvin. Onnistuin kehittämään ohjelmointi taitojani vahvemmiksi. Vaikka tekoäly oli läsnä projektissa esimerkiksi haastavampien kirjastojen kanssa, kaikki vaiheet on silti pyritty tekemään itse. Koodin rakenne ja vaiheet onnistuttiin myöskin suunnittelemaan itsenäisesti ja saamaan toimivaksi kokonaisuudeksi. Ohjelmoinnissa jonkin verran auttoi myös aikaisempi empiirisen projektin ohjelmakoodi, jonka perusteista tämäkin ohjelma sai apua.

SQL osaamiseni kehittyi myös jonkin verran. Tietokannan luontilause oli tuttu konsepti jo aiemmin mutta sen luomiseen sai hieman varmuutta. Kyselyjen tekemisen harjoittelu jatkuu tulevaisuudessa, mutta sitäkin on jo saatu kehitettyä huomattavasti.